

Logistic Regression

DSC 305

February 24, 2025

What is machine learning?

Overview

Learning

- Originated in AI:

***"An agent is learning
if it improves its performance on future tasks
after making observations about the world."***

- How? Suppose we have example data: **input-output pairs**
- Objective: Learn **function** that predicts **output** given **input**

Tasks (based on feedback)

- **Unsupervised** learning (e.g., clustering)
- **Reinforcement** learning: rewards/punishments
- **Supervised** learning: learns function from inputs to outputs
- Hybrids (**semi-supervised** learning) also exist

Supervised Learning

Let's start here!

Supervised Learning

- Given **training set** of N examples:

$$(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$$

- Assume each y_i generated by **unknown function** $y = f(\mathbf{x})$
- Goal: **Discover function h that approximates true function f .**

$$\hat{y} = h(\mathbf{x}) \approx f(\mathbf{x})$$

- Function h is **hypothesis**
- Notation: $\mathbf{x}$ columns in DF, y is last column, superscript $y^{(i)}$ row index

Supervised Learning

- We want h to **perform well** on new examples
- **Test-train split** (Science: **test your hypothesis!**)
- **Generalizes well** if it **predicts** or infers new examples correctly
- True function f may be **stochastic**
- In that case, we are learning $P(Y|\mathbf{x})$.
 - “Probability of the outcomes conditioned on observing feature values $\mathbf{x}$.”

Outputs and Learning

- When output Y is finite, the learning problem is **classification**
 - e.g., {sunny, cloudy, rainy}
- If Y is continuous, then the problem is called **regression**
 - e.g., tomorrow's temperature

Logistic Regression

Estimate probability of y given observations $\mathbf{x}$

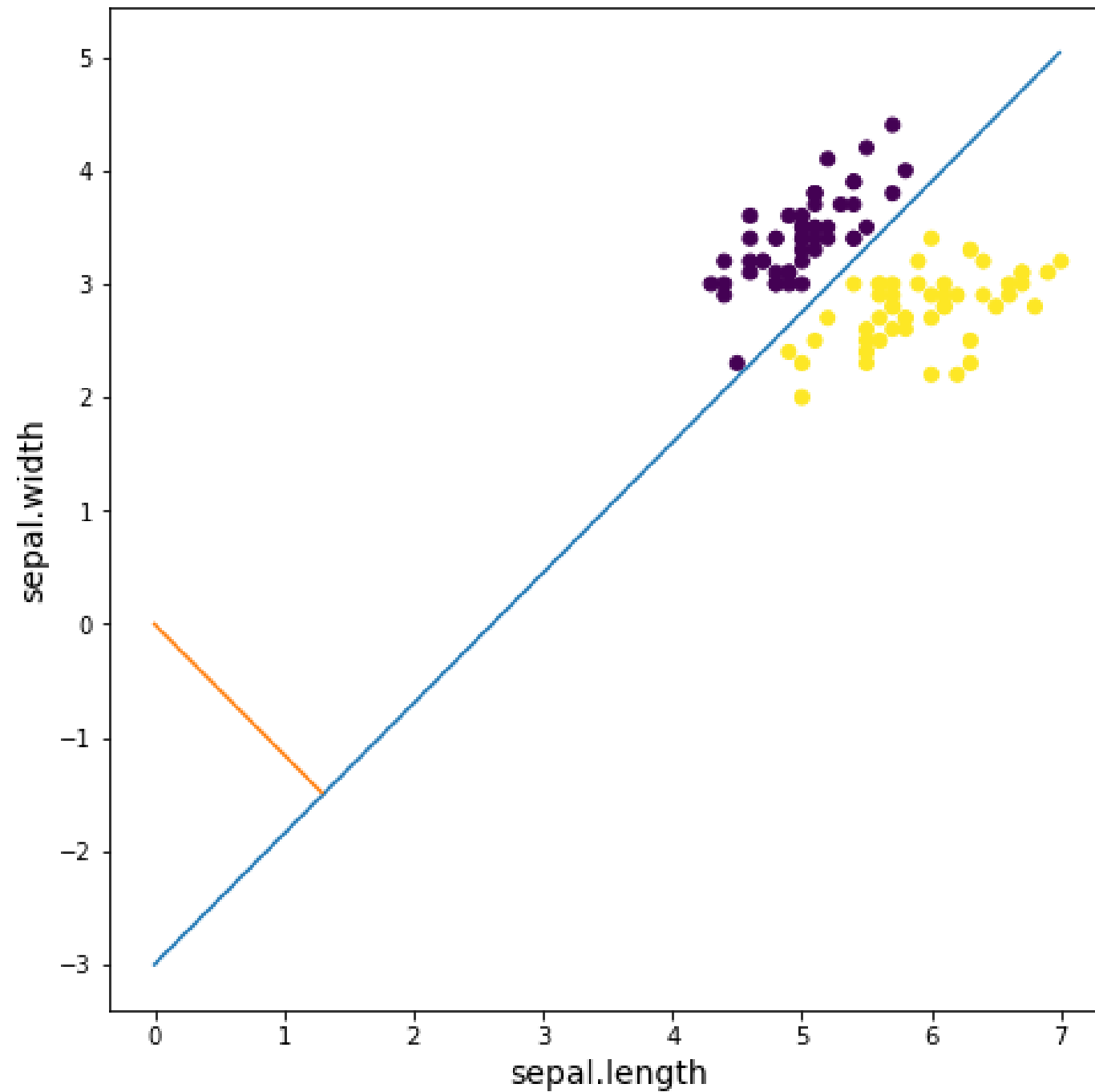
Logistic Regression

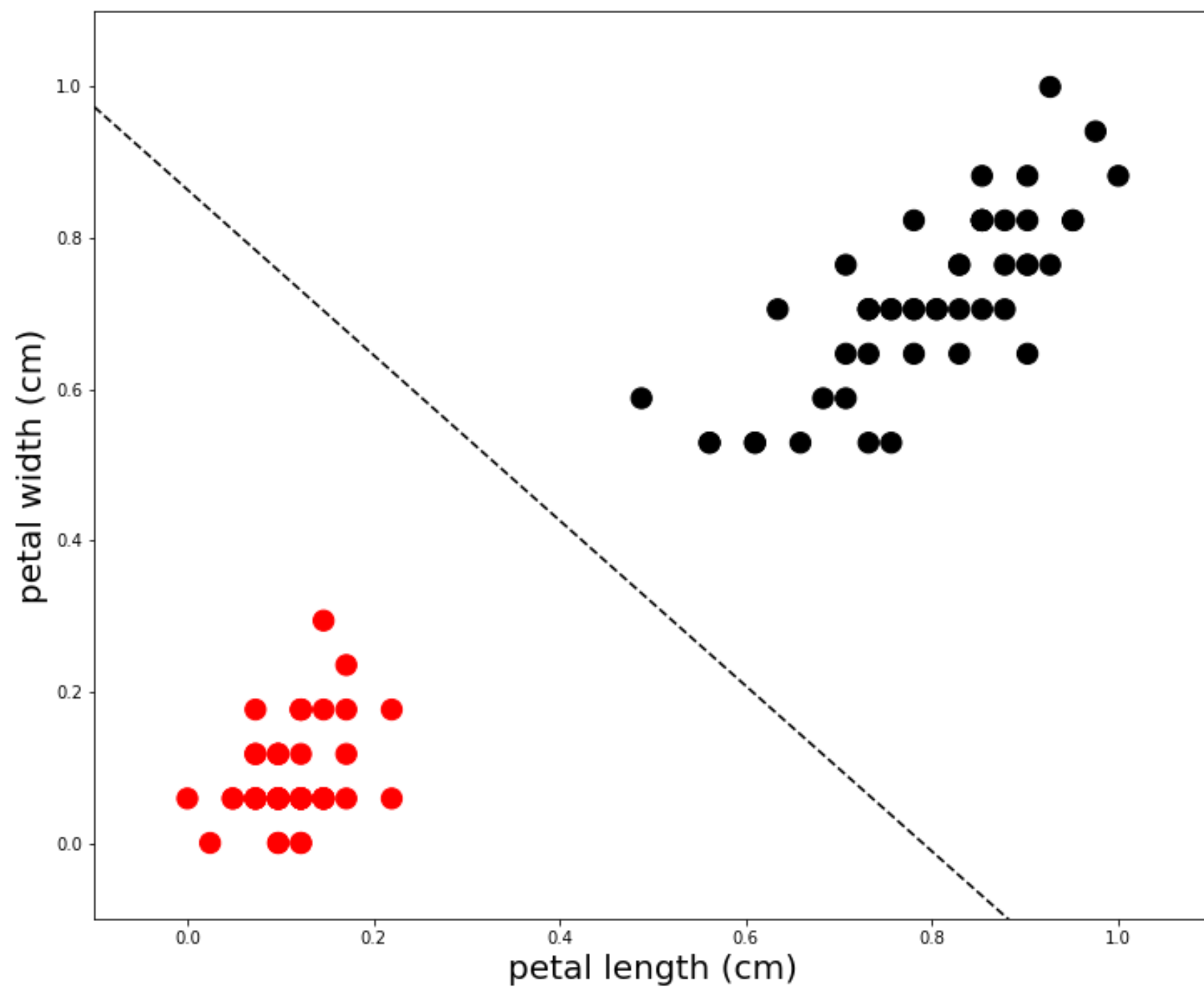
- Name misleading. Used for **classification**, not regression!
 - Problem: Predict class label.
 - Categorical, not numeric. Can be binary or multinomial.
- Examples:
 - Identify the person in this photograph.
 - Will this customer "churn" (switch to another company)?
- **Probabilistic** approach to classification.
 - Fit **linear model** to feature space.
 - Big idea: Plot data, find a direction (vector $\mathbf{w}$) of increasing probability
 - Decision boundary is hyperplane with probability 0.5



Iris Data Set

Linear model using Iris data





Scenario

- **Multidimensional** feature space (continuous or categorical features).
- **Outcome is discrete** (not continuous). **Binary** or **multinomial**.
- Plausible that **linear decision boundary** will give accurate results.
 - Caution: Linear does not necessarily mean a line!
 - It means a linear combination of features
 - Geometric interpretation: hyperplane in feature space
 - Often will still be some overlap: probabilistic boundary

Why not just use linear regression?

- In principle we could.
- Treat binary label as 0.0 or 1.0.
- Prediction will be a float, but we can round (threshold = 0.5).
- We would minimize mean-squared error.

[-0.11195616 1.73309429 -0.4959663 1.70005246 0.02160356 -0.14031562]
4.52466081992959

Mean squared error (MSE): 0.02

Variance (R²) score: 0.98

Predictions:

37.5	0.0	0.0	1.0	0.0	0.0	2	2.000	2.026
37.6	0.0	0.0	1.0	1.0	1.0	2	2.000	1.896
36.0	0.0	1.0	0.0	0.0	0.0	0	-0.000	-0.002
37.9	0.0	0.0	1.0	1.0	0.0	2	2.000	2.003
37.0	0.0	0.0	1.0	1.0	1.0	2	2.000	1.964
.	.	.	.	.	.	.	.	.
40.0	1.0	1.0	1.0	1.0	1.0	3	3.000	2.865

Accuracy is 100.0 percent

Why not just use linear regression?

- Unfortunately, results often inaccurate.
- Sensitive to outliers.
- Hypothesis can be less than 0 or greater than 1.
- The predicted “y” value isn’t a probability.
- We can do better!

Expressing Probability

- $P(y = 1) = p$
- $P(y = 0) = 1 - p = q$
- Can express probability of p as:

	Notation	Range Equivalents		
Standard probability	p	0	0.5	1
Odds	p / q	0	1	$+\infty$
Log odds (logit)	$\ln(p/q)$	$-\infty$	0	$+\infty$

Model

- Model consists of **vector** w in n -dimensional feature space
- For point $\mathbf{x}$ in feature space, **project it** onto $\mathbf{w}$
- This converts it to real number z in range $-\infty$ to $+\infty$

$$z = w_0 + \mathbf{w} \cdot \mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{j=1}^n w_jx_j$$

- Map to the range 0 to 1 using the logistic function

$$p = \frac{1}{1 + e^{-z}}$$

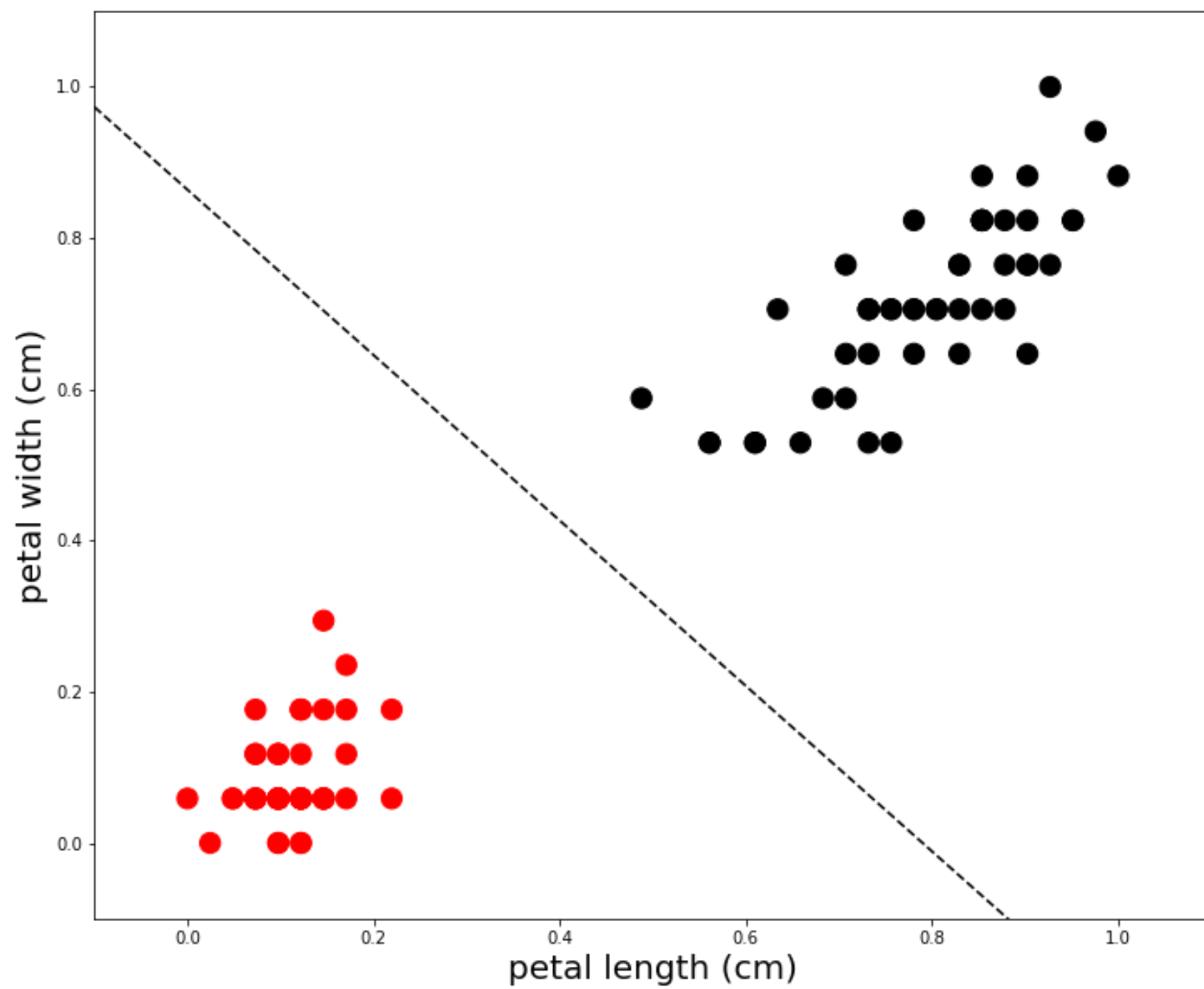
- Overall, logistic regression maps point $\mathbf{x}$ to a value in $[0, 1]$

Takeaway

- Multiply features by coefficients $w_j x_j$
- Add together, plus intercept term w_0
- Call this z
- Probability that it is the “positive” class $y = 1$ is:

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-z}}$$

- Weight vector $\mathbf{w}$ is the direction of increasing probability.
- Can choose any threshold as your decision boundary (usu. 50/50).



Interpretation

Can interpret prediction as:

- **Probability** of class membership
- **Class assignment**, by applying **threshold** to probability (e.g., 0.5)
- Threshold represents **decision boundary** in feature space

Fitting a Model

- Estimate weights $\vec{w}$ and bias (intercept) w_0
- Typically uses numerical methods (similar to Newton's)
- Goes through a lot of iterations until it converges
- Details beyond the scope of this course (but covered in DSC 395 😊)
- Many packages provide a choice of solvers (check the manual)
- In sklearn there is also a choice of method for multi-class fitting

Tradeoffs

- Method is well-understood
 - Transparent results (weights)
 - No (well, relatively few) assumptions about underlying distribution
 - Works for multi-class problems
 - Training is usually very fast
 - Highly resistant to overfitting
 - Simple method: if in doubt, try it first for classification problems
- Assumes linear boundary
 - Many problems are not linearly separable

Jupyter Notebook Demo (logistic.ipynb)

- **Scale data** with MinMaxScaler (or may not **converge**)
- sklearn.linear_model.**LogisticRegression**
 - There is also sklearn.linear_model.LinearRegression. Don't mix them up!

```
In [8]: model = LogisticRegression(solver='lbfgs', multi_class='multinomial')
        model.fit(X, y)
        w = model.coef_
        w0 = model.intercept_
        print('w =', w)
        print('w0 =', w0)|
```

```
w = [[-1.40492627  1.56002082 -2.86063347 -2.78713597]
      [ 0.26135757 -1.43723031  0.41839556 -0.47843834]
      [ 1.1435687  -0.12279052  2.44223791  3.2655743 ]]
w0 = [ 2.20369771  1.22915069 -3.4328484 ]
```